Beyond Fixed Learning Rates: Globalization Strategies for Adaptive Step Size in Deep Reinforcement Learning

by

Oliver Diamond

A thesis submitted in partial fulfillment of the requirements for the degree of

Master of Science

Department of Computing Science

University of Alberta

Abstract

Add the abstract here.

Preface

Add the preface here.

Acknowledgements

Add ack here.

Contents

Abstract ii

Preface iii

Acknowledgements iv

List of Tables vii

List of Figures viii

List of Algorithms ix

1 Introduction 1

2 Problem Formulation and Background 3

2.1 Armijo and Nonmonotone Line Searches 4

3 Parallel Argmin Line Search (PALS) 5

4 Empirical Study 7

4.1 Experiment Setup 7

4.2 PPO Control in Brax MuJoCo 7

4.3 The effect of batch size 9

References 10

A Line Search Algorithms **15**

B PPO Control in MuJoCo **16**

List of Tables

List of Figures

4.1 PPO control with Adam and RMSProp in MuJoCo over 20 seeds. 8

4.2 PPO control with Adam and RMSProp with different line search methods, aggregated across MuJoCo. 8

4.3 The effect of batch size on performance in Humanoid of different line search methods: PALS, backtracking Armijo and backtracking Nonmonotone. (a) Mean return of PPO with RMSProp using a batch size of 1024 and 64. (b) Step sizes for the actor and critic in PPO with RMSProp for batch size 1024 and 64 over 20 seeds. 9

B.1 PPO control with Adam in MuJoCo over 20 seeds. 16

B.2 PPO control with RMSprop in MuJoCo over 20 seeds. 17

B.3 PPO control with SGD in MuJoCo over 20 seeds. 17

List of Algorithms

1 Parallel Argmin Line Search 15

Chapter 1

Introduction

Deep reinforcement learning (RL) algorithms are notoriously sensitive to hyperparameters, often requiring manual tuning on each new environment to avoid poor performance [1, 2]. Setting the step size hyperparameter is particularly challenging because good values are often determined by the optimization landscape, and good performance is typically achieved by changing it over time. Even modern optimizers like Adam and RMSprop suffer from step size sensitivity with deep RL algorithms [3, 4, 5]. In fact, both algorithms can completely fail to learn in non-stationary prediction settings with a constant step size [6, 7], which is concerning due to the non-stationarity inherent in policy optimization.

In practice, tuning step size is common for deep RL algorithms. Perhaps the most common approach is a grid search, where a wide range of hyperparameter combinations are tested. Grid searches can be effective but typically require the parallelism allowed for by a simulator, limiting their applicability. Further these broad hyperparameter searches are not generally possible for agents deployed in real-world settings such as industrial control [8, 9], where testing multiple hyperparameter configurations can be time-consuming or dangerous. Finally, grid searches typically find a fixed step size that works well on average throughout learning and across runs – not a step size tailored to a specific agent on a specific time step. Bayesian and evolutionary methods can more intelligently and efficiently identify good fixed hyperparameter settings than grid searches, but nevertheless suffer from similar drawbacks in terms of the need for extensive computation and applicability to simulators [10]. Fixed default step sizes exist for common benchmarking suites like MuJoCo Playground and the Arcade Learning Environment (ALE). (cite: ALE, MuJoCo Playground) However, different repositories use slightly different settings and recent work has demonstrated that these defaults 1) typically do not work across tasks, 2) are not optimized for individual environments, and 3) are sensitive to small changes [11, 2, 1]. This is a major obstacle to deployment of these deep RL methods in new domains, especially by non RL experts.

Meta-gradient approaches frame hyperparameter tuning as optimization, iteratively learning hyperparameters to optimize a hyperparameter objective. Numerous meta-learning approaches specialized to learning step sizes have been explored over the years, including Incremental Delta-Bar-Delta [12], Stochastic Meta Descent [13], Autostep [14], AdaGain [15], and similar extensions to Temporal-Difference (TD) learning [16, 17, 18]. In deep RL, meta-gradients have been used to learn effective discount factors, λ-returns, update targets for value functions and policies, full objective functions, intrinsic rewards, and other hyperparameters [19, 20, 21, 22, 23, 24, 25, 26, 27]. Meta-gradients have even been used to learn the optimization algorithm itself [28, 29]. These methods are promising but often significantly increase training time and have sensitive hyper-hyperparameters that require tuning.

In the optimization literature, a more common method of selecting step sizes is to use a backtracking line search [30]. Line searches search along a given update direction u for a step size which reduces the objective value sufficiently. Line searches are agnostic to the way in which u is produced, often only requiring it to be descent directions on the objective. In this way, they are general and can be layered on top of popular modern optimizers such as RMSprop [31] to automate step size selection and adapt the step size over the course of an experiment. Although originally designed for deterministic objectives, line search methods have been extended to the stochastic setting in supervised learning and work in the tabular RL case, demonstrating impressive empirical performance on popular benchmarks and enjoying robust theoretical guarantees [32, 33, 34, 35, 36, 37, 38, 39, 40]. Outside a minor role in the TRPO algorithm [41], to the best of our knowledge, stochastic line search methods have not been successfully combined with modern optimizers in deep RL.

In this paper, we investigate line search in RL prediction and control settings. We introduce a new algorithm that is more effective for RL, that uses a parallel search to avoid sequential backtracking and common initialization heuristics [33, 34], and allows for use of a more effective selection criterion based on the step size that minimizes the objective the most. We prove that this approach converges linearly under an interpolating condition matching the rate of previous (stochastic) line-search methods [33, 34], when layered on-top of standard optimizers like RMSProp or AMSgrad, a convergent variant of Adam. We provide extensive suite of experiments comparing several line search methods in Atari 2600 and MuJoCo in both prediction and control, highlighting that our algorithm performs amongst the best across these settings. Finally, we outline a specific failure case of line searches on Atari 2600, namely that they may exacerbate convergence to poor, sharp minima.

Chapter 2

Problem Formulation and Background

We formalize the agent-environment interaction as a Markov Decision Process (MDP), defined by the tuple $(\mathscr{S}, \mathscr{A}, \mathscr{R}, \mathscr{P}, d_0, \gamma)$, where $\mathscr{S}$ and $\mathscr{A}$ denote the state and action spaces, respectively. The reward function $\mathscr{R} : \mathscr{S} \times \mathscr{A} \times \mathscr{S} \to \mathbb{R}$ provides a (possibly stochastic) scalar signal following each transition. The dynamics function $\mathscr{P} : \mathscr{S} \times \mathscr{A} \to \Delta_{\mathscr{S}}$ maps state-action pairs to distributions over subsequent states, where $\Delta_{\mathscr{X}}$ denotes the set of distributions over $\mathscr{X}$. The initial state is sampled from the starting distribution $d_0 \in \Delta_{\mathscr{S}}$, and $\gamma \in [0, 1)$ is the discount factor. The environment starts in a state $S_0 \sim d_0$. At each step t, the agent observes the current state S_t and selects an action $A_t \sim \pi(\cdot \mid S_t)$ according to its policy $\pi : \mathscr{S} \to \Delta_{\mathscr{A}}$. The environment transitions to a new state $S_{t+1} \sim \mathscr{P}(S_t, A_t)$ and provides the agent with a reward $R_{t+1} = \mathscr{R}(S_t, A_t, S_{t+1})$. The agent's objective is to maximize the expected return G_t, defined as the cumulative discounted future reward $G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$. The task of finding a policy to maximize the expected return $\mathbb{E}_{\pi, \mathscr{P}, d_0}[G_0]$ is referred to as the *control* problem.

To learn a policy which maximizes return, algorithms often rely on value functions which quantify the utility of states or actions in terms of reward. The state-value function $v_\pi : \mathscr{S} \to \mathbb{R}$ is the expected return when starting in some state and following policy π, $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ and similarly the action-value function is $q_\pi : \mathscr{S} \times \mathscr{A} \to \mathbb{R}$ is $q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$. These functions are estimated with parameterized functions $v_{\boldsymbol{\phi}}$ and $q_{\boldsymbol{w}}$ where $\boldsymbol{\phi}$ and $\boldsymbol{w}$ are learnable parameters. The task of accurately estimating these values for a fixed policy is known as the *prediction* problem. Monte Carlo methods for prediction provide unbiased targets for learning these value functions using full trajectory rollouts, but can suffer from high variance. Temporal Difference (TD) methods can reduce variance by bootstrapping targets from value estimates, though they may introduce some bias.

2.1 Armijo and Nonmonotone Line Searches

Consider the stochastic minimization problem $x^\star = \text{argmin}_{x\in\mathbb{R}^n} f_k(x)$, where f_k is an unbiased stochastic estimate of an objective function f. Iterative algorithms typically update the current estimate x_k by choosing an update direction u_k and taking a step in this direction, setting $x_{k+1} = x_k + \eta_k u_k$ where $\eta_k > 0$ is a scalar step size. While η_k is often fixed or decayed, recent research has pivoted toward adaptive step size selection to improve convergence and robustness.

A prominent class of adaptive methods is line search [30, 42, 33, 38, 34, 43, 37]. These methods search for an optimal η_k along the direction u_k. In particular, the stochastic Armijo line search selects an η_k that satisfies the *stochastic Armijo condition* [33]:

$$f_k(x_k + \eta_k u_k) \leq f_k(x_k) + c\eta_k \langle \nabla f_k(x_k), u_k \rangle, \tag{2.1}$$

where $c \in (0, 1)$ is a hyperparameter and $\langle \nabla f_k(x_k), u_k \rangle < 0$. Line searches are flexible, and can easily be layered atop any optimizer that produces an update vector u_k that satisfies this property [30, 38, 35].

Backtracking is used to get η_k by evaluating the sequence $\eta_k^{(i)} = \eta_0 \beta^i$ for $i = 0, \ldots, B$ where $\eta_0 > 0$ is some initial step size. The algorithm sets $\eta_k = \eta_k^{(i)}$ using the smallest i satisfying Equation 2.1, typically falling back to $\eta_k^{(B)}$ if no candidate is found. The sequential cost of B backtracking evaluations led to heuristics to optimize the initial guess η_0. Notably, [33] proposed to initialize each iteration k of the line search from $\eta_k \delta^{n/b}$, where $\delta > 1$, n is the dataset size, and b is the batch size. [34] suggested using Polyak step sizes for initialization.

Despite its theoretical guarantees, the Armijo condition requires a monotonic decrease that can be restrictive, resulting in minuscule step sizes [30]. To mitigate this, non-monotone line searches relax the requirement of a strict decrease at every iteration and instead simply require that progress be made overall [44, 34, 43, 45, 46, 47, 48]. This is often accomplished by replacing the $f(x_k)$ term in Equation 2.1. Non-monotone line searches often choose larger step sizes than Armijo line searches, potentially finding flatter, more robust minima [43]. Many non-monotone conditions exist; we use one that requires progress over an exponential moving window of iterates [44, 34]:

$$\begin{gathered} f_k(x_k + \eta_k u_k) \leq C_k + c\eta_k \langle \nabla f_k(x_k), u_k \rangle \quad \text{where} \\ C_k = \max\left\{\tilde{C}_k, f_k(x_k)\right\} \qquad \tilde{C}_k = \frac{\xi Q_k C_{k-1} + f_k(x_k)}{Q_{k+1}} \qquad Q_{k+1} = \xi Q_k + 1 \end{gathered} \tag{2.2}$$

where again $c \in (0, 1)$ and $\langle \nabla f_k(x_k), u_k \rangle < 0$. Here, C_k is a bias-corrected exponential moving average of function values, except that when $f_k(x_k) > \tilde{C}_k$, the term $C_k = f_k(x_k)$ subsumes the weight for the past function values.

Chapter 3

Parallel Argmin Line Search (PALS)

We propose a framework for integrating line search techniques into RL to adaptively determine step sizes for both value function approximation and policy optimization. As discussed, a primary hurdle to adopting line searches is the sequential backtracking procedure. One simple insight is that the backtracking procedure is in fact paralellizable. Using modern hardware accelerators, the entire step size search can be executed in parallel. This alters the design space: we no longer must rely on conservative initializations or incremental heuristics for computational efficiency. Instead, we can recover a panoptic view of the local objective landscape. By searching over a sufficiently large set of step sizes, the algorithm can better capture the local geometry and can achieve greater progress.

Rather than accepting the first step size that satisfies the Armijo condition as is typical with sequential backtracking line searches, we propose a more aggressive selection strategy. We select the step size that most minimizes f_k along u_k while still satisfying the stochastic Armijo condition:

$$\eta_k = \arg\min_{\eta \in \mathcal{H}_k} f_k(x_k + \eta u_k) \tag{3.1}$$

where $\mathcal{H}_k$ includes the smallest step size and the candidate step sizes that satisfy Equation 2.1:

$$\mathcal{H}_k = \left\{ \eta_k^{(i)} \;\middle|\; i \in \{0, 1, \ldots, B\},\ f_k(x_k + \eta_k^{(i)} u_k) \leq f_k(x_k) + c\eta_k^{(i)} \langle \nabla f_k(x_k), u_k \rangle \right\} \cup \left\{ \eta_0 \beta^B \right\} \tag{3.2}$$

We call this the *Parallel Argmin Line Search* (PALS), with pseudocode in Algorithm 1 in Appendix A. Note that in our ablations we also tested a variant that uses the non-monotone condition, but found it to perform worse.

The requirement $\langle \nabla f_k(x_k), u_k \rangle < 0$ disqualifies many popular optimizers, such as Adam [49]. Letting $g_k = \nabla f_k(x_k)$ and $\beta_1, \beta_2 > 0$, Adam computes $u_k = -A_k^{-1} m_k$ where $A_k^2 = \text{diag}(\beta_2 g_{k-1} + (1 - \beta_2) g_k^2)$ is a diagonal preconditioner and $m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k$ is a momentum term.

This momentum term causes difficulties since $\langle \nabla f_k(x_k), m_k \rangle_{A_k^{-1}}$ is not guaranteed to be negative. Recently, [36] motivated a new way to combine Adam with line search methods by dropping m_k in the Armijo condition when searching for a step size η_k:

$$f_k(x_k - \eta_k A_k^{-1} \nabla f_k(x_k)) \leq f_k(x_k) - c\eta_k \left\| \nabla f_k(x_k) \right\|_{A_k^{-1}}, \tag{3.3}$$

while still updating with $u_k = -A_k^{-1} m_k$. We adopt this approach when using PALS with Adam. Importantly, the entire line search remains parallelizable.

Chapter 4

Empirical Study

4.1 Experiment Setup

To ensure consistency, we maintained identical hyperparameters across all experiments unless otherwise noted. MuJoCo and Brax experiments use the standard PPO MLP: two hidden layers of 64 units with tanh activations [50]. We used a minibatch size of 1024. Fixed step size baselines for SGD, RMSprop, and Adam were tuned via grid search over $\eta \in \{10^{-5}, \ldots, 10^{-1}, 0.5\}$. Following [51], we selected the step size η that maximized aggregate performance across all seeds and environments after normalizing training loss (rescaling 5th and 95th percentiles to 0 and 1 respectively). Line searches always started from $\eta_0 = 1$ and performed $B = 30$ backtracking steps with $\beta = 0.7$. We set $c = 0.1$ for Armijo variants [33] and $c = 0.5$ for Non-monotone variants [34].

4.2 PPO Control in Brax MuJoCo

We now examine the performance of line search methods in control with PPO. We use line search for both the parameterized policy and state-value network. In this section, we omit Heuristic Armijo as applying it in the online setting is not straightforward. We measure the performance of all methods over 3 million steps on 5 environments from the Brax MuJoCo suite [52] with 20 random seeds. For baselines, we linearly annealed step sizes over time as is common with PPO [50] and examined both the default PPO batch size of 64 and batch size 1024 (denoted PPO-64 and PPO-1024). We tuned the initial step sizes for the linearly annealed PPO-64 and PPO-1024 separately. All other hyperparameter and network architecture settings are set according to [50].

Figure 4.2 shows the return for all methods, with Adam and RMSProp over 3 million environment interactions. Results for SGD are shown in Appendix B. We can see that overall PALS performs consistently well for the two optimizers, doing slightly worse than backtracking Armijo

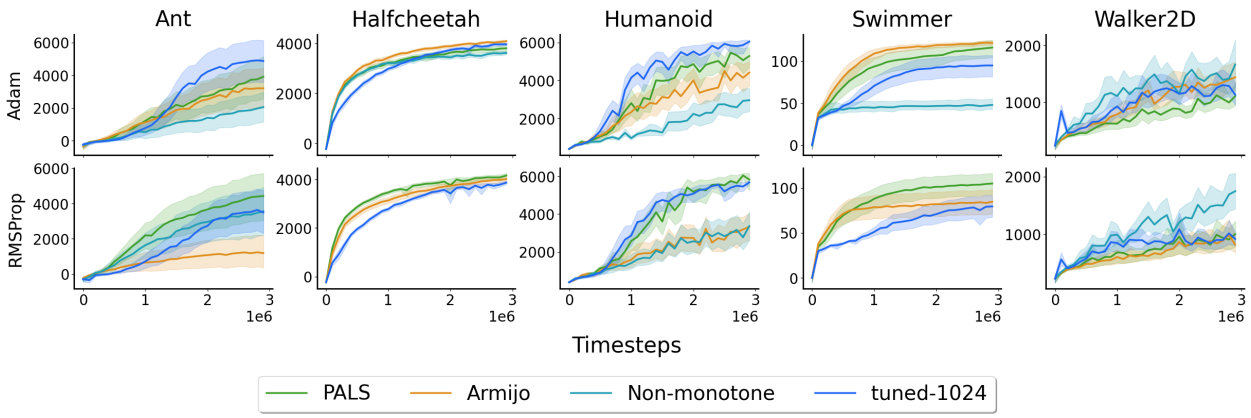

Figure 4.1: PPO control with Adam and RMSProp in MuJoCo over 20 seeds.

and a fixed step size with Adam but notably better than backtracking Armijo and a fixed step size with RMSProp.

We also provide the learning curves in Figure 4.1, for a closer look. PALS roughly matched or outperformed a fixed step size in 4 of 5 environments with Adam and all environments with RMSProp and SGD. Backtracking Armijo typically performed worse than a fixed step size for all three base optimizers, while backtracking non-monotone was typically much worse with Adam (where it selected larger critic step sizes than other methods) and better with RMSProp and SGD. Overall, PALS can reliably match the performance of a tuned baseline across diverse settings where standard backtracking approaches struggle.

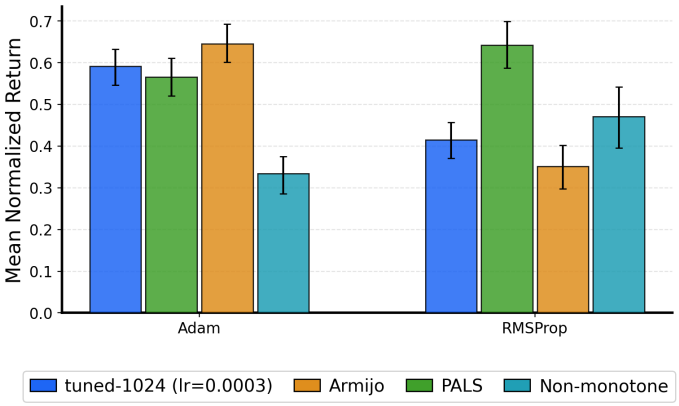

Figure 4.2: PPO control with Adam and RMSProp with different line search methods, aggregated across MuJoCo.

In the full-batch, non-convex, supervised setting, line search algorithms have been shown to select highly conservative step sizes and fail to reach the edge of stability (EOS) [53], resulting in slow convergence to sharp, poor-performing local minima. However, we do not observe this behavior with the Armijo line search in the online control setting, perhaps because the implicit regularization of stochastic updates may alleviate EOS undershooting [53]. In fact, the strong performance of PALS relative to regular backtracking may be attributed to selecting more conservative step sizes in the RL control setting, where stochastic

estimates of non-stationary objectives can be high-variance.

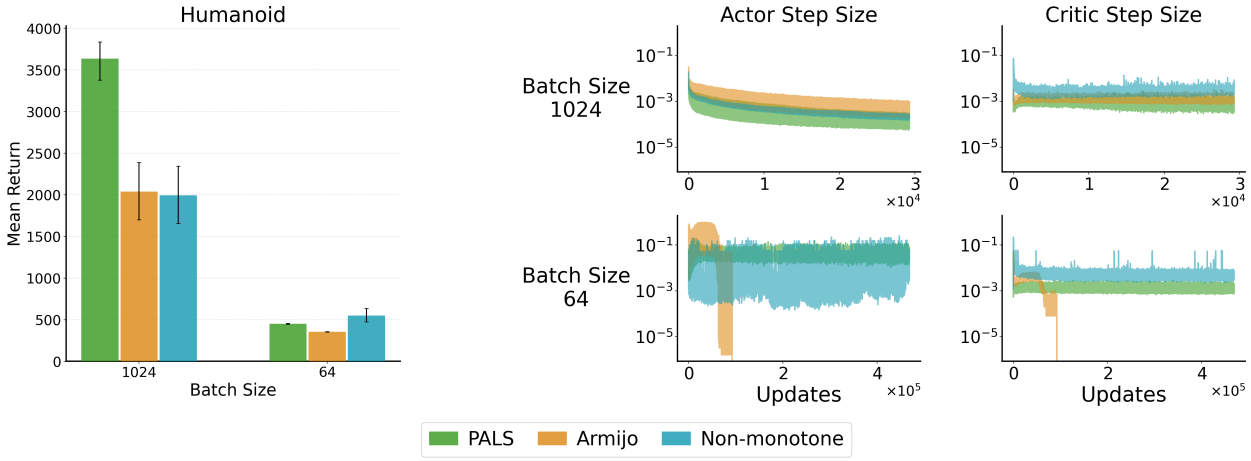

Figure 4.3: The effect of batch size on performance in Humanoid of different line search methods: PALS, backtracking Armijo and backtracking Nonmonotone. (a) Mean return of PPO with RMSProp using a batch size of 1024 and 64. (b) Step sizes for the actor and critic in PPO with RMSProp for batch size 1024 and 64 over 20 seeds.

4.3 The effect of batch size

To examine the effect of batch size on the performance of the line search methods in the control setting, Figure 4.3(a) compares the mean return in Ant with batch size 1024 and 64 with RMSProp. Similar to what has been observed in supervised learning, a larger batch size greatly improves the control performance of all line search methods [54], with nearly all failing to learn with small batch size. Figure 4.3(b) shows that with small batch size, the accepted step sizes can be larger with higher variance, particularly for the actor. This result holds for SGD and Adam as well.

One other interesting note is that PALS also improved robustness to the batch size choice. We obtained consistent performance across all settings with a fixed batch size of 1024 for PALS. This was not the case for a fixed step size. PPO-64 achieved better performance than PPO-1024 and the line search methods in 3 of 5 environments, but it failed in Humanoid. By contrast, PALS achieved robust performance across all environments with a single batch size.

References

[1] T. Eimer, M. Lindauer, and R. Raileanu, "Hyperparameters in Reinforcement Learning and How To Tune Them," in *Proceedings of the International Conference on Machine Learning*. PMLR, 2023, pp. 9104–9149.

[2] J. Adkins, M. Bowling, and A. White, "A Method for Evaluating Hyperparameter Sensitivity in Reinforcement Learning," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2024.

[3] B. Ellis, M. T. Jackson, A. Lupu, A. D. Goldie, M. Fellows, S. Whiteson, and J. Foerster, "Adam on Local Time: Addressing Nonstationarity in RL with Relative Adam Timesteps," *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, vol. 37, pp. 134 567–134 590, 2024.

[4] H. Donâncio, A. Barrier, L. F. South, and F. Forbes, "Dynamic Learning Rate for Deep Reinforcement Learning: A Bandit Approach," *arXiv preprint arXiv:2410.12598*, 2024.

[5] S. M. Jordan, S. Neumann, J. E. Kostas, A. White, and P. S. Thomas, "The Cliff of Overcommitment with Policy Gradient Step Sizes," *Reinforcement Learning Journal*, 2024.

[6] A. Jacobsen, M. Schlegel, C. Linke, T. Degris, A. White, and M. White, "Meta-Descent for Online, Continual Prediction," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.

[7] T. Degris, K. Javed, A. Sharifnassab, Y. Liu, and R. Sutton, "Step-Size Optimization for Continual Learning," *arXiv preprint arXiv:2401.17401*, 2024.

[8] J. Luo, C. Paduraru, O. Voicu, Y. Chervonyi, S. Munns, J. Li, C. Qian, P. Dutta, J. Q. Davis, N. Wu *et al.*, "Controlling Commercial Cooling Systems using Reinforcement Learning," 2022, arXiv:2211.07357.

[9] M. K. Janjua, H. Shah, M. White, E. Miahi, M. C. Machado, and A. White, "GVFs in the Real World: Making Predictions Online for Water Treatment," *Machine Learning*, vol. 113, no. 8, pp. 5151–5181, 2024.

[10] J. Parker-Holder, R. Rajan, X. Song, A. Biedenkapp, Y. Miao, T. Eimer, B. Zhang, V. Nguyen, R. Calandra, A. Faust *et al.*, "Automated Reinforcement Learning (AutoRL): A Survey and Open Problems," *Journal of Artificial Intelligence Research*, 2022.

[11] A. Patterson, S. Neumann, R. Kumaraswamy, M. White, and A. White, "Cross-environment hyperparameter tuning for reinforcement learning," *Reinforcement Learning Journal*, 2024.

[12] R. S. Sutton, "Adapting Bias by Gradient Descent: an Incremental Version of Delta-Bar-Delta," in *National Conference on Artificial Intelligence*, 1992.

[13] N. Schraudolph, "Local Gain Adaptation in Stochastic Gradient Descent," in *International Conference on Artificial Neural Networks*, 1999.

[14] A. R. Mahmood, R. S. Sutton, T. Degris, and P. M. Pilarski, "Tuning-Free Step-Size Adaptation," in *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2012.

[15] A. Jacobsen, M. Schlegel, C. Linke, T. Degris, A. White, and M. White, "Meta-descent for Online, Continual Prediction," in *AAAI Conference on Artificial Intelligence*, 2019.

[16] A. Kearney, V. Veeriah, J. B. Travnik, R. S. Sutton, and P. M. Pilarski, "TIDBD: Adapting Temporal-difference Step-sizes Through Stochastic Meta-descent," 2018.

[17] K. Young, B. Wang, and M. E. Taylor, "Metatrace Actor-Critic: Online Step-Size Tuning by Meta-gradient Descent for Reinforcement Learning Control," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence*, 2019.

[18] M. Thill, "Temporal Difference Learning Methods with Automatic Step-Size Adaptation for Strategic Board Games: Connect-4 and Dots-and-Boxes," 2015, cologne University of Applied Sciences, Master's Thesis.

[19] Z. Xu, H. P. van Hasselt, and D. Silver, "Meta-gradient reinforcement learning," *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, vol. 31, 2018.

[20] Z. Xu, H. P. van Hasselt, M. Hessel, J. Oh, S. Singh, and D. Silver, "Meta-Gradient Reinforcement Learning with an Objective Discovered Online," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2020.

[21] J. Oh, M. Hessel, W. M. Czarnecki, Z. Xu, H. P. van Hasselt, S. Singh, and D. Silver, "Discovering Reinforcement Learning Algorithms," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2020.

[22] L. Kirsch, S. van Steenkiste, and J. Schmidhuber, "Improving Generalization in Meta Reinforcement Learning using Learned Objectives," in *International Conference on Learning Representations*, 2020.

[23] Y. Wang and T. Ni, "Meta-SAC: Auto-Tune the Entropy Temperature of Soft Actor-Critic via Metagradient," *ICML Workshop on Automated Machine Learning*, 2020.

[24] Z. Zheng, J. Oh, M. Hessel, Z. Xu, M. Kroiss, H. Van Hasselt, D. Silver, and S. Singh, "What Can Learned Intrinsic Rewards Capture?" in *Proceedings of the International Conference on Machine Learning*, 2020.

[25] C. Bonnet, L. Midgley, and A. Laterre, "Debiasing Meta-Gradient Reinforcement Learning by Learning the Outer Value Function," in *Workshop on Meta-Learning*, 2022.

[26] T. Zahavy, Z. Xu, V. Veeriah, M. Hessel, J. Oh, H. P. van Hasselt, D. Silver, and S. Singh, "A Self-Tuning Actor-Critic Algorithm," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2020.

[27] Z. Zheng, J. Oh, and S. Singh, "On Learning Intrinsic Rewards for Policy Gradient Methods," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2018.

[28] M. Andrychowicz, M. Denil, S. Gomez, M. W. Hoffman, D. Pfau, T. Schaul, B. Shillingford, and N. de Freitas, "Learning to Learn by Gradient Descent by Gradient Descent," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2016.

[29] Q. Lan, A. R. Mahmood, S. Yan, and Z. Xu, "Learning to Optimize for Reinforcement Learning," *Reinforcement Learning Journal*, 2024.

[30] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., ser. Springer Series in Operations Research and Financial Engineering. New York, NY: Springer, 2006.

[31] G. Hinton, "Lecture 6.5-RMSProp: Divide the Gradient by a Running Average of its Recent Magnitude," *COURSERA: Neural networks for machine learning*, vol. 4, no. 2, p. 26, 2012.

[32] M. Lu, M. Aghaei, A. Raj, and S. Vaswani, "Towards principled, practical policy gradient for bandits and tabular mdps," *arXiv preprint arXiv:2405.13136*, 2024.

[33] S. Vaswani, A. Mishkin, I. Laradji, M. Schmidt, G. Gidel, and S. Lacoste-Julien, "Painless Stochastic Gradient: Interpolation, Line-Search, and Convergence Rates," in *Advances in Neural Information Processing Systems*, 2019.

[34] L. Galli, H. Rauhut, and M. Schmidt, "Don't be so Monotone: Relaxing Stochastic Line Search in Over-Parameterized Models," *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2023.

[35] B. Shea and M. Schmidt, "Why Line Search when you can Plane Search? SO-Friendly Neural Networks allow Per-Iteration Optimization of Learning and Momentum Rates for Every Layer," 2024.

[36] S. Vaswani, I. Laradji, F. Kunstner, S. Y. Meng, M. Schmidt, and S. Lacoste-Julien, "Adaptive gradient methods converge faster with over-parameterization (but you should do a line-search)," *Annual Workshop on Optimization for Machine Learning*, 2020.

[37] C. Fox and M. Schmidt, "Glocal Smoothness: Line Search can really help!" in *Workshop on Optimization for Machine Learning*, 2024.

[38] S. Vaswani and R. Babanezhad, "Armijo Line-search Can Make (Stochastic) Gradient Descent Provably Faster," in *International Conference of Machine Learning*, 2025.

[39] C. Fan, G. Choné-Ducasse, M. Schmidt, and C. Thrampoulidis, "BiSLS/SPS: Auto-tune Step Sizes for Stable Bi-level Optimization," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2023.

[40] X. Jiang and S. U. Stich, "Adaptive SGD with Polyak stepsize and Line-search: Robust Convergence and Variance Reduction," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2023.

[41] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust Region Policy Optimization," in *Proceedings of the International Conference on Machine Learning*, 2015.

[42] L. Armijo, "Minimization of Functions having Lipschitz Continuous First Partial Derivatives," *Pacific Journal of Mathematics*, 1966.

[43] C. Fox, L. Galli, M. Schmidt, and H. Rauhut, "Nonmonotone Line Searches Operate at the Edge of Stability," in *Workshop on Optimization for Machine Learning*, 2024.

[44] H. Zhang and W. W. Hager, "A Nonmonotone Line Search Technique and Its Application to Unconstrained Optimization," *SIAM Journal on Optimization*, 2004.

[45] L. Grippo, F. Lampariello, and S. Lucidi, "A Nonmonotone Line Search Technique for Newton's Method," *SIAM Journal on Numerical Analysis*, vol. 23, 1986.

[46] Y.-H. Dai, "A Nonmonotone Conjugate Gradient Algorithm for Unconstrained Optimization," *Journal of Systems Science and Complexity*, vol. 15, 2002.

[47] Q. Huynh, L. Lasdon, and J. Plummer, "Nonmonotone Line Search for Minimax Problems," *Journal of Optimization Theory and Applications*, vol. 76, 1993.

[48] E. G. Birgin, J. M. Martínez, and M. Raydan, "Nonmonotone Spectral Projected Gradient Methods on Convex Sets," *SIAM Journal on Optimization*, vol. 10, 2000.

[49] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proceedings of the International Conference on Learning Representations*, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[50] S. Huang, R. F. J. Dossa, A. Raffin, A. Kanervisto, and W. Wang, "The 37 Implementation Details of Proximal Policy Optimization," 2022, iCLR Blog Track.

[51] S. Neumann, S. Lim, A. G. Joseph, Y. Pan, A. White, and M. White, "Greedy Actor-Critic: A New Conditional Cross-Entropy Method for Policy Improvement," in *Proceedings of the International Conference on Learning Representations*, 2023.

[52] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem, "Brax - A Differentiable Physics Engine for Large Scale Rigid Body Simulation," 2021. [Online]. Available: http://github.com/google/brax

[53] V. Roulet, A. Agarwala, J.-B. Grill, G. Swirszcz, M. Blondel, and F. Pedregosa, "Stepping on the Edge: Curvature Aware Learning Rate Tuners," in *Proceedings of the International Conference on Advances in Neural Information Processing Systems*, 2024.

[54] V. Roulet, A. Agarwala, and F. Pedregosa, "On the Interplay Between Stepsize Tuning and Progressive Sharpening," *OPT2023 Workshop on Optimization for Machine Learning*, 2023.

Appendix A

Line Search Algorithms

In this section, we show the line search algorithm used in this work. Algorithm 1 shows the Parallel Argmin Line Search (PALS).

Algorithm 1 Parallel Argmin Line Search

Require: (Possibly stochastic) Loss function $f : \mathbb{R}^n \to \mathbb{R}$, Initial point $x_0 \in \mathbb{R}^n$, $c \in (0, 1)$, Initial step size $\eta_0 > 0$, Contraction factor $\beta \in (0, 1)$, Backtracking steps $B \in \mathbb{N}$, Convergence tolerance $\epsilon > 0$

Initialize iteration counter $k \leftarrow 0$
while $\|\nabla f(x_k)\| > \epsilon$ **do**
    Compute descent direction u_k ▷ e.g., $u_k = -\nabla f(x_k)$
    Set $\mathcal{H}_k = \left\{ \eta_k^{(i)} = \eta_0 \beta^i \mid i = 0, 1, \ldots, B \text{ and } f(x_k + \eta_k^{(i)} u_k) \leq f(x_k) + \eta_k^{(i)} c \langle \nabla f(x_k), u_k \rangle \right\}$
    if $\mathcal{H}_k$ is empty **then**
        $\eta_k^* = \eta_0 \beta^B$ ▷ Fallback if Armijo condition is not met
    else
        $\eta_k^* = \operatorname{argmin}_{\eta \in \mathcal{H}_k} f(x_k + \eta u_k)$
    Update point: $x_{k+1} = x_k + \eta_k^* u_k$
    Increment counter: $k \leftarrow k + 1$
return x_k

Appendix B

PPO Control in MuJoCo

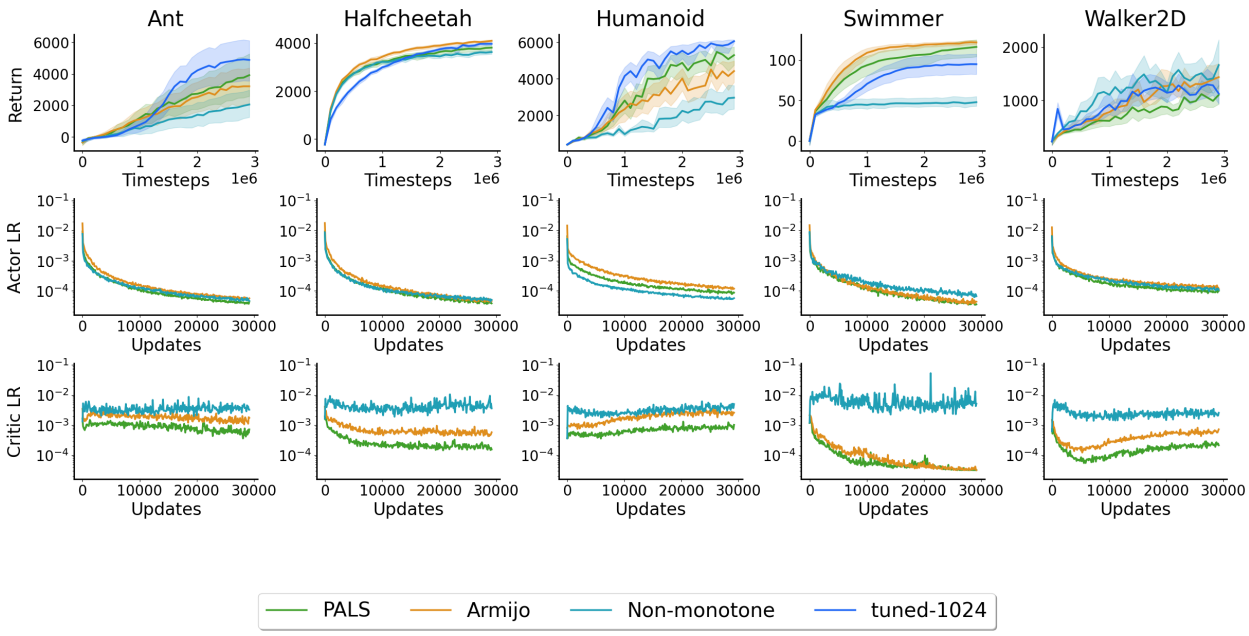

Figure B.1: PPO control with Adam in MuJoCo over 20 seeds.

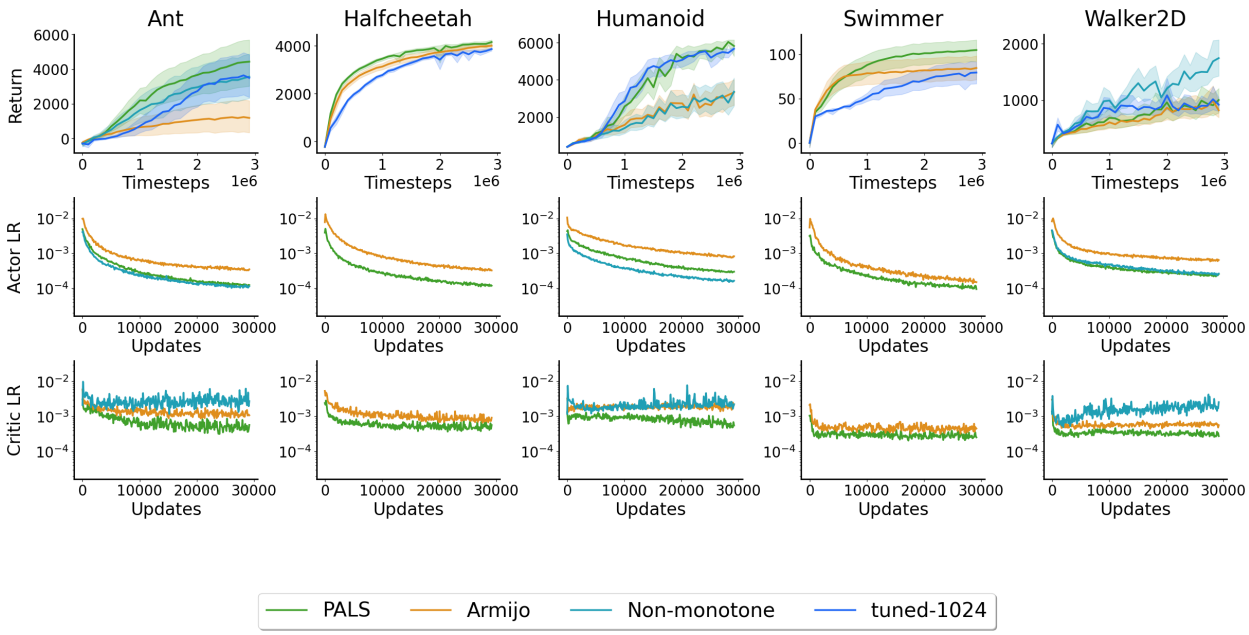

Figure B.2: PPO control with RMSprop in MuJoCo over 20 seeds.

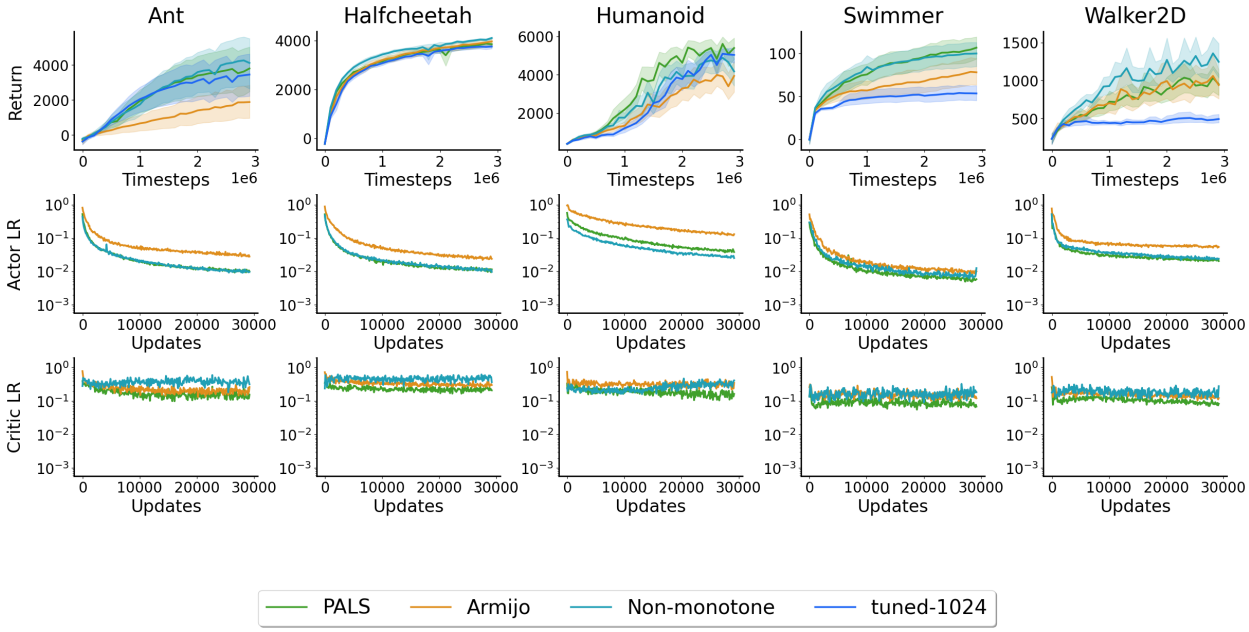

Figure B.3: PPO control with SGD in MuJoCo over 20 seeds.